

Monitoring, Metrics, and Alerts

1. Key Observability Metrics

To maintain high availability and identify performance degradation before it impacts end users, we track five critical system metrics across microservices, caches, queues, and databases:

Metric Category	Target Component	Measurement & Purpose	Healthy Threshold
Booking Latency & Throughput	BookingService (APIs)	Tracks requests per second (RPS) alongside p50, p95, and p99 response times for /bookings/lock and /bookings/confirm.	p95 < 2.0s, p99 < 3.5s
Lock Acquisition Rate	Redis Distributed Locks	Ratio of successful vs failed SETNX calls. Spikes in failures highlight extreme seat contention during blockbuster releases.	Failure rate < 15%
Queue Lag	Kafka (booking_events)	Number of unprocessed messages across consumer groups (NotificationService, LoyaltyService). Detects consumer bottlenecks.	Lag < 500 messages
Cache Hit Ratio	Redis Cache Cluster	Percentage of movie catalog and showtime reads served by Redis without querying PostgreSQL.	Hit Ratio > 85%
Payment Failures	External Gateways	Number of payment declines, network timeouts, and gateway 5xx errors per minute. Triggers circuit breaker failover.	Failure rate < 2%

2. Observability Tooling Stack

- Prometheus:** Periodically scrapes operational time-series metrics from internal microservice endpoints, Redis exporters, PostgreSQL statistics collectors, and Kafka JMX metrics.
- Grafana:** Provides centralized real-time dashboards visualizing traffic volume, p95/p99 latency heatmaps, lock acquisition rates, and error rate spikes.
- ELK Stack (Elasticsearch, Logstash, Kibana):** Aggregates and indexes structured JSON application logs. Every user request carries a unified request_id header across the API Gateway, BookingService, and Kafka consumers to enable end-to-end distributed tracing.

3. Service Level Objectives (SLOs) & Alert Rules

Primary Booking Latency SLO

- Service Level Objective:** 95% of seat lock and booking confirmation requests must complete in < 2.0 seconds during normal and peak operating hours.
- SLI Formula:** (Requests completing in < 2.0s / Total booking requests) * 100 >= 95%
- Alert Rule:** If the 95th percentile (p95) latency exceeds 2.0 seconds for more than 3 consecutive minutes, Prometheus Alertmanager fires a **P2 (High Priority)** notification to the on-call engineering team.

Secondary Queue Processing Lag SLO

- Service Level Objective:** Asynchronous ticket delivery via NotificationService must maintain a Kafka consumer lag of **under 1,000 messages**.
- Alert Rule:** If consumer lag exceeds 1,000 messages for longer than 5 minutes, an automated alert fires and triggers horizontal pod auto-scaling on notification worker instances.

System Availability SLO

- Service Level Objective:** 99.9% monthly availability for core ticket browsing and checkout paths (excluding user-side payment card declines).

4. Alert Routing & Incident Response

Pipeline: Metric Threshold Exceeded

1. Prometheus detects metric breach (e.g., p95 booking latency > 2.0s for 3 min)

2. Alertmanager groups correlated alerts to prevent alert fatigue

3. Notification dispatched to PagerDuty and Slack engineering incident channel

4. Automated mitigation triggered (e.g., auto-scale BookingService pods or open Payment Circuit Breaker)

5. On-call engineer investigates root cause via Grafana dashboard and ELK logs